Assignment 2

Data Structures (CSC211)

Instructor: Mam. Humera Faisal

M. Athar Abbas

SP25-BSE-082 | Section A

Call Center Agent Rotation System using Circular Linked List

1. Problem Description

In a call center, incoming customer calls need to be evenly distributed among available agents to ensure fairness and efficiency. To achieve this, agents are arranged in a circular sequence, where each agent gets a turn to handle a call in order. Once the last agent receives a call, the next incoming call should go back to the first agent again — making it a round-robin distribution.

The task is to simulate this call distribution system using a Circular Linked List (CLL) data structure.

2. Solution (Code)

The following C++ program implements the circular linked list-based call center agent rotation system:

```
#include <iostream>
using namespace std;

struct agent{
    int id;
    bool busy; //1 for busy , 0 for free.
    agent(int id, bool status) {
        this->id = id;
        this->busy = status;
    }
};

struct node {
    agent* ag;
    node* next;
};

struct circularList{
    int count;
    node* head;
};

node* getLast(circularList* l) {
    if(l -> head == NULL) {
```

```
        return NULL;
    }
    node* last = l -> head -> next;
    while(last -> next != l -> head) {
        last = last -> next;
    }
    return last;
}

void add(circularList* l , agent* a) {
    node* n = new node();
    n -> ag = a;
    if(l -> head == NULL) {
        l -> head = n;
    } else {
    node* last = getLast(l);
    last -> next = n;
    }
    n -> next = l -> head;
    l->count++;
}

void printList(circularList* l){
    node* temp = l -> head;
    while(temp != NULL) {
        cout << "Agent : " <<temp -> ag -> id << "  => " ;
        temp -> ag -> busy == 1 ? cout << "Busy\n" : cout << "Free\n";
        temp = temp -> next;
        if (temp == l -> head)
            return;
    }
}


void printStatus(agent* a) {
    string id = to_string(a -> id);
    if (a -> busy) {
        cout << "Agent " + id + " is on the call now." << endl;
    } else {
        cout << "Agent " + id + " is free." << endl;
    }
}

void updateStatus(circularList* l, int match_id){
    node* temp = l -> head;

    while(temp != NULL) {

        if(temp -> ag -> id == match_id){
            temp -> ag -> busy = true;
        } else {
        temp -> ag -> busy = false;
        }
```

```
        temp = temp -> next;
        if (temp == l -> head)
            break;
    }
}

void simulate(circularList* l) {
    if(l -> head == NULL) {
        cout << "Cant simulate , list empty." << endl;
        return;
    }

    int busyId = 1;
    node* first;
    node* last = getLast(l);

    for(int i = 0 ; i < 6 ; i++) {
        updateStatus(l, busyId);
        first = l -> head;
        while(first != last) {
            printStatus(first -> ag);
            first = first -> next;
        }
        printStatus(last -> ag);

        if((i + 1) % l -> count == 0){
            busyId = 1;
        } else {
            busyId++;
        }
        cout << endl;
    }
}



int main() {
    cout << "Athar Abbas" << endl;
    cout << "SP25-BSE-082" << endl;
    cout << "Section - A" << endl;
    cout << "============" << endl << endl;
    circularList* lst = new circularList();

    agent* a1 = new agent(1,false);  //everyone's free for now.
    agent* a2 = new agent(2,false);
    agent* a3 = new agent(3,false);
    agent* a4 = new agent(4,false);
    agent* a5 = new agent(5,false);

    add(lst ,a1);
    add(lst, a2);
    add(lst, a3);
    add(lst, a4);
    add(lst, a5);
```

```
    simulate(lst);
    return 0;
}
```

3. Output

The following is the program output showing 6 call simulation rounds across 5 agents in a round-robin fashion:

```
Athar Abbas
SP25-BSE-082
Section - A
============

Agent 1 is on the call now.
Agent 2 is free.
Agent 3 is free.
Agent 4 is free.
Agent 5 is free.

Agent 1 is free.
Agent 2 is on the call now.
Agent 3 is free.
Agent 4 is free.
Agent 5 is free.

Agent 1 is free.
Agent 2 is free.
Agent 3 is on the call now.
Agent 4 is free.
Agent 5 is free.

Agent 1 is free.
Agent 2 is free.
Agent 3 is free.
Agent 4 is on the call now.
Agent 5 is free.

Agent 1 is free.
Agent 2 is free.
Agent 3 is free.
Agent 4 is free.
Agent 5 is on the call now.

Agent 1 is on the call now.
Agent 2 is free.
Agent 3 is free.
Agent 4 is free.
Agent 5 is free.
```